

research

Phase 0 Research — Provable Model Alias Availability

Date: 2026-09-08 | Plan: [plan.md](#) | Spec: [spec.md](#)

Phase 0 exists to resolve the plan’s NEEDS ATTENTION items before any repair is designed. The principal unknown was: **why are 11 providers pinned in overrides.json and still reported orphaned?** FR-020 commits to repairing all 20 orphans, and the repair shape depends entirely on the cause, so planning a repair before knowing it would be the §11.4.102 violation this phase exists to prevent.

Decision: the orphan cause is NOT YET KNOWN.
Four candidates are eliminated by measurement.

Rationale. Each candidate below was tested, not reasoned about. Four are eliminated. Reporting a repair plan now would require guessing between the survivors, which §11.4.6 forbids.

#	Candidate cause	Verdict	Evidence
1	Credentials missing or rotated	ELIMINATED	All 29 provider records name a key variable and all 29 of those variables are SET in the environment. Compared by hash throughout; no value read or printed (§11.4.10).
2	Model catalogue stale, absent or incomplete	ELIMINATED	models.dev.cache.json is 4.5 MB, parses, carries 213 providers, was 23.6 h old against a 24 h TTL, and contains 6 of the 7 orphan key variables checked. HUGGINGFACE_API_KEY is genuinely absent from it

#	Candidate cause	Verdict	Evidence
			— a separate, single-provider issue, not the class cause.
3	Missing pins in <code>overrides.json</code>	ELIMINATED as the class cause	Running the resolver directly: 10 of the 11 pinned-yet-orphaned providers DO resolve (deepseek, huggingface, hyper, kc-for-coding, kilo, nvidia, opencode, openrouter, poe, zai-coding-plan). Only kc-for-coding2 does not, and it shares <code>ApiKey_Kimi</code> with kc-for-coding — the same one-record-per-key-variable mechanism as the 9 unpinned records. So pins are necessary for the 9; they are not what is failing for the 11.
4	Key variables invisible to <code>present_key_vars</code>	ELIMINATED — refuted by experiment	<code>present_key_vars</code> greps <code>\$CMA_KEYS_FILE</code> for literal <code>NAME=</code> assignments only, and cannot see variables supplied by a file that keys file <code>SOURCES</code> . Measured: the operator's <code>api_keys.sh</code> carries exactly ONE literal assignment while the records reference <code>FOURTEEN</code> , so 13 of 14 are invisible to that grep. That looked decisive. It is not: re-running <code>list-all</code> against a temporary keys file containing all 14 names as literal (obviously fake) assignments produced

#	Candidate cause	Verdict	Evidence
			byte-identical counts — 6 failed, 20 orphaned, 1 stale:verified, 2 verified. Visibility does not drive orphan status.

Alternatives still open (the honest remainder). The leading surviving candidate is that `list-all` computes orphan status from a **cached or stored resolved set** rather than resolving live — which would explain precisely why changing the resolver’s inputs changed nothing. That is testable and is the first task of the next research pass. It is recorded as a hypothesis, not a finding.

Why candidate 4 is written up despite being wrong. It produced a striking, publishable-looking measurement — “13 of 14 key variables are invisible” — and the arithmetic that followed from it fit the observed 20 orphans closely enough to be persuasive. It was still wrong. Recording the elimination is worth more than deleting it, because the next person to measure key-variable visibility will otherwise reach the same dead end and believe it (§11.4.7, §11.4.112).

Instrument note (§11.4.273) — this phase is the anchor’s first real use

The anchor requiring a positive and negative control on any decision-bearing census was minted from this session’s earlier errors, and it earned itself twice within this single research pass:

1. The first resolver invocation returned one record and no ids. The **positive control failed** (`helixllm-gateway` absent), so the instrument was treated as the suspect rather than the system — and it was: `--keys` takes a comma-separated list of key-variable NAMES, and it had been handed a file PATH, which it dutifully treated as a variable name. Without the control the reading would have been “the resolver resolves nothing”, which is alarming, plausible, and false.
2. The corrected run’s positive control ALSO failed, for a different reason: `helixllm-gateway` is out of the resolver’s declared scope by design — Helix providers are added by the shell (`claude-providers.sh:248`, “`providers_resolve.py` stays pure”). **A positive control must lie inside the instrument’s declared scope**; one chosen outside it produces a false instrument-is-broken signal. Re-run with `deepseek`, an in-scope catalogue provider, the control passed and the measurement became trustworthy.

A third case is the reason candidate 4 is in the eliminated column rather than the findings column: `DEEPSEEK_API_KEY` was used as a positive control for key-file

visibility when **its presence was the very question under test**. A control whose truth is what you are measuring is not a control.

Decision: honest SKIP-with-reason for three mandatory test types (§11.4.169 / §11.4.3)

Rationale. §11.4.169 requires the full test-type set where the domain warrants it. Three do not apply here, and fabricating them would produce green suites asserting nothing — the PASS-bluff the anchor exists to prevent.

Test type	Disposition	Reason
DDoS / load-flood	SKIP-with-reason	The subject is a single-model gateway on loopback with one operator. There is no flood surface; a synthetic one would measure the harness.
Scaling	SKIP-with-reason	No horizontal dimension exists — one model, one host, one process. Re-evaluate if a second serving host is added.
Chaos (infrastructure failure injection)	PARTIAL	Process-death and network-fault injection against the gateway ARE meaningful and in scope. Disk-full and OOM injection are not: the component holds no significant state and allocates per request.

Unit, integration, e2e, full-automation, security, stress, concurrency, race, memory and benchmark all apply and are in scope for tasks .md.

Decision: cross-platform parity is a declared, tracked gap (§11.4.81)

Rationale. The Darwin* date branches and the uname -s cache exist in the code and are unexercised — this host is Linux only. Deleting them would break macOS

outright; claiming them verified would be a \$11.4 bluff. The honest position is a declared gap carried in Complexity Tracking until a Darwin host is available, stated rather than implied by an all-PASS table.

Alternatives considered. Emulation was rejected: a macOS date implementation running under emulation proves the flag parsing, not the platform, and would produce evidence stronger-looking than it is.

T005 diagnosis-gate advance — measured 2026-09-08 (read-only)

Run against the live host with the toolkit tree quiescent (0 uncommitted files), which is the condition the earlier pass deliberately waited for.

Two more candidate causes ELIMINATED

Environment / key-variable presence — ELIMINATED. `resolve_records()` builds - keys from `present_key_vars`, i.e. the key variables exported in the INVOKING shell, so the resolved set is environment-dependent and this was a live hypothesis. Measured: 45 key variables present in the working shell, and 45 after sourcing the operator's key file — identical. The environment is not the differentiator.

The export-ownership escape misfiring — ELIMINATED. `cma_find_orphans` skips any record carrying a `CMA_PROVIDER_SOURCE` marker, so a bug there would orphan export-owned records wrongly. Measured across all 28 orphans: ZERO carry the marker, i.e. the escape is not being bypassed. Control: a non-orphan DOES carry it, proving the detector is not simply blind.

The set is larger than the baseline recorded

28 orphaned entries in `status.json` (53 total), not 20. The baseline figure has grown, so it is drift, not a fixed number — worth re-measuring rather than citing.

The 28 split cleanly by a property the baseline did not record: - **20 carry an .env** — these are the “presented as available, refuse on use” class the specification is about. - **8 are status-only**, with no .env at all — pure leftovers, and a different remedy (prune) from the 20.

Phase 0's causal model is REFUTED for 9 of the 20

Phase 0 held that the 9 unpinned orphans share a key variable with siblings, so catalogue matching — which yields exactly one record per key variable — can never reach them; and that the 11 pinned ones are orphaned for an

undetermined reason. That reads the pinned/unpinned split and the shared/unique-key split as the same partition. Cross-tabulated, they are not:

	shares key var	unique key var
pinned	4	7
unpinned	7	2

The largest single cell is **pinned AND unique key var (7)** — neither proposed cause applies to it. Adding the 2 unpinned-with-unique-key, **9 of the 20 have a key variable no sibling shares**, so “one record per key variable” cannot be their explanation. Three key variables account for all the sharing: OPENROUTER_API_KEY (5), NVIDIA_API_KEY (4), ApiKey_Kimi (2).

What this leaves (§11.4.6)

The cause for the 9 unique-key orphans is UNDETERMINED. It is NOT credentials, NOT catalogue staleness, NOT missing pins, NOT key-variable visibility, NOT the shell environment, NOT the ownership escape, and NOT key-variable sharing — seven causes eliminated with measurement rather than argument. The surviving hypothesis is unchanged and still untested: that the resolver simply emits no record for these ids, which would make orphaning a correct verdict on a real gap rather than a reporting defect. Testing it needs the resolver’s own output diffed against the status keyset, which needs the models.dev cache path — that did not resolve in this pass (cma_models_dev_cache returned empty from a non-interactive shell) and is the immediate next step.

Honest boundary. Everything above is a property of the RECORDS. None of it establishes whether any of these 20 aliases would work if invoked; that is the end-to-end question T005 gates and it remains open.